

Unit-II. Process Synchronization (Concurrency Control), Deadlocks

Introduction:

We know that, program in execution is called 'process' & it is categorized into two types on the basis of synchronization and these are given below:

- 1) Independent Process
- 2) Cooperative Process

1) Independent Processes:

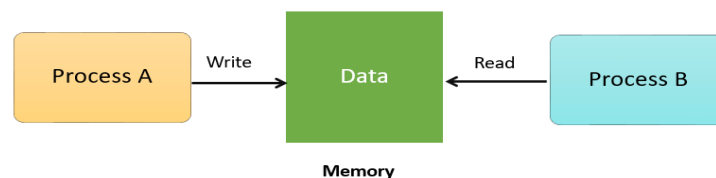
- Two processes are said to be independent if the execution of one process does not affect the execution of another process.

2) Cooperative Processes:

- Two processes are said to be cooperative if the execution of one process affects the execution of another process.
- In this case, the execution of one process affects execution of another process because these processes share something. Like they share variable, memory, code, resources (CPU, Printer, Scanner etc).
- These cooperative processes needs to be synchronized to generate correct result after execution.

Process Synchronization:

- It is the task of coordinating the execution of processes in such a way that no two processes can have access to the same shared data and resources.
- The concept of process Synchronization is introduced to handle problems that arises while multiple process executes.
- It is a procedure that is involved to preserve the appropriate order of execution of cooperative processes.
- Process Synchronization is mainly needed in a multi-processor / multi-process system where multiple processes are running together, and more than one processes try to access the same shared resource or any data at the same time. This can lead to the inconsistency of shared data.
- So the change made by one process necessarily reflected when other processes accessed the same shared data. To avoid this type of inconsistency of data, the processes need to be synchronized with each other.
- Process Synchronization is a mechanism that deals with the synchronization of processes.
- It controls the execution of processes running concurrently to ensure that consistent results are produced.
- In order to synchronize the processes, there are various synchronization mechanisms.
- Consider following diagram, Process A changing the data in a memory location while another Process B is trying to read the data from the same memory location. There is a high probability that data read by the second process will be wrong.



Example-

The following illustration shows how inconsistent results may be produced if multiple processes execute concurrently without any synchronization.

- Process P1 and Process P2 are cooperative process and are in ready queue.
- They share common variable 'temp'
- These process run parallel in a system.
- Let, Process P1 executes first since it comes first in ready queue.

int temp= 5	
P1	P2
<pre>{ int x; x=temp; x++; sleep(1); temp=x; }</pre>	<pre>{ int y; y=temp; y--; sleep(1); temp=y; }</pre>

Execution of P1 and P2 are as follow-

- Execution of P1 process-
 - First 'x' has assigned value of shared variable 'temp' which is 5.
 - x++ increments value of x by 1. Therefore value of x becomes 6.
 - Sleep(1) executes, this pauses process P1 for 1 unit of time.
 - Due to pause of P1, CPU scheduler allocates next process P2 to CPU from ready queue for execution. Here the situation where context switching is happen.
- Now, Execution of P2 process starts-
 - First 'y' has assigned value of shared variable 'temp' which is 5.
 - y-- decrements value of y by 1. Therefore value of y becomes 4.
 - Sleep(1) executes, this pauses process P2 for 1 unit of time.
 - Due to pause of P2, now CPU scheduler allocates process P1 to CPU from ready queue for execution. Here the situation where context switching is happen.
- Now again Execution of P1 process starts-
 - The execution of P1 start from next statement where last time it goes sleep() (Because in PCB execution is also maintained).
 - temp=x executes it assigns value of 'x' to 'temp'. Now value of 'temp' becomes **6**.
 - After that no any statement to execute for P1 & hence it terminates.
 - After termination of P1, CPU scheduler allocates process P2 to CPU from ready queue for execution.
- Now again Execution of P2 process starts-
 - The execution of P2 start from next statement where last time it goes sleep().
 - temp=y executes it assigns value of 'y' to 'temp'. Now value of 'temp' becomes **4**.
 - After that no any statement to execute for P2 & hence it also terminates.
 - At last, value of 'temp' shared variable became '4' which is wrong.
- Because we add 1 and subtract 1 from 5 (Initially temp=5). But, by adding and subtracting 1 from 5 it remains as it is i.e. 5.
- Here, we lastly got value of shared 'temp' variable as **4** which is wrong.
- This is because of these cooperative processes P1 and P2 that runs parallel **without** synchronization.
- It is clear from here that inconsistent results may be produced if multiple processes execute concurrently without any synchronization.

Therefore, such cooperative processes needs to be synchronized.

Race Condition:

- When more than one process is either executing the same code or accessing the same memory or any shared variable; In that condition, there is a possibility that the output or the value of the shared variable will be wrong so for that purpose all the processes are doing the race to say that my output is correct. This condition is commonly known as a 'race condition'.
- As several processes, access and process the same data in a concurrent manner and due to which the outcome depends on the particular order of execution of processes in which the access of data takes place which is explained in following example.
- Mainly this race condition is a situation that may occur inside the critical section.
- Race condition in the critical section happens when the result of multiple thread execution differs according to the order of execution of instructions of processes.
- But this race condition in critical sections can be avoided if the critical section is treated as an atomic operation/instruction (Atomic operation are those operations that finish their tasks as a whole i.e. no other interruption is allowed before its completion). Proper thread synchronization using locks or atomic variables can also prevent race conditions.
- Race condition is a situation where the final output produced depends on the execution order of instructions of different processes.
- (Example – Students given a mathematical problem to solve. All students solve problem concurrently and produce answer or result. Here, all students are in race to solve the mathematical problem. Some student produce right answer and some produce wrong answer because they all trying to solve problem concurrently)

Example-

The following illustration shows race condition that produced inconsistent result if multiple processes execute concurrently without any synchronization.

- Process P1 and Process P2 are cooperative process and are in ready queue.
- They share common variable 'temp' having assigned value 2.
- These process run parallel in a system.

int temp= 2;			
P1		P2	
{		{	
int x;		int y;	
1)	x=temp-1;	1)	y=2*temp;
2)	temp=2*x;	2)	temp=y-1;
}		}	

Here, different execution order of the instructions of P1 and P2 produce different results.

Case-01: Consider execution order of the instructions: P1(1), P1(2), P2(1), P2(2)

In this case, Final value of 'temp' = 3

Case-02: Consider execution order of the instructions: P2(1), P2(2), P1(1), P1(2)

In this case, Final value of 'temp' = 4

Case-03: Consider execution order of the instructions: P1(1), P2(1), P2(2), P1(2)

In this case, Final value of 'temp' = 2

From above three cases, it is clear that final value of 'temp' shared variable is depends of execution order of instructions in cooperative processes P1 and P2 that runs without synchronization.

- In concurrent programming, concurrent processes access to shared resources can lead to unexpected or inaccurate behavior, so parts of the program where the shared resource is accessed need to be protected to avoid the concurrent access. This protected section is the 'critical section or critical region'.
- Critical section is a section of the program where a process access the shared resources during its execution.
- Critical section cannot be executed by more than one process at a time.
- When one process enters in critical section and execute this part, no other process can enter it i.e. for other process this section is blocked for a while.
- Different codes or processes may consist of the same variable or other resources that need to be read or write but whose results depend on the order of execution of instructions in such cases critical section is needed.
- Typically, the critical section accesses a shared resource, such as a memory, a peripheral device, or a network connection, that would not operate correctly in the context of multiple concurrent accesses.

The diagram illustrates a mutual exclusion algorithm. It features two processes, Process 1 and Process 2, each represented by a vertical sequence of three blue circles. A central red circle represents the critical section. Arrows show the flow of execution. A label 'Critical Section Required' points to the red circle. Dashed arrows at the bottom indicate the continuation of the processes.

- ### Process A:

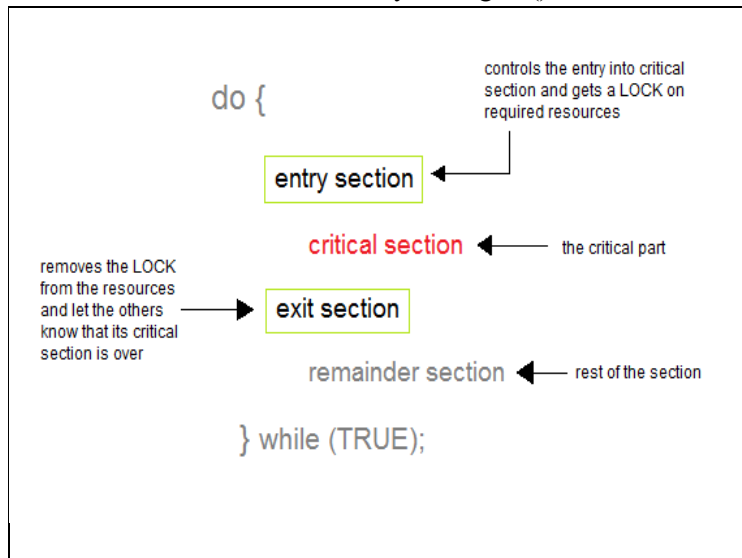
Process B:

- In cases like these, a critical section is important. In the above case, if Process A needs to read the updated value of x, executing Process A and Process B at the same time may not give correct results. To prevent this, variable x is protected by a critical section. First, B gets the access to the critical section. Once B finishes writing the value x, then A gets the access to the critical section and variable x can be read.
- By carefully controlling which variables are modified inside and outside the critical section, concurrent access to the shared variable are prevented.

- A critical section is typically used when a multi-threaded program must update multiple related variables without a separate thread, making inconsistent changes to that data. In such a situation, a critical section may be used to ensure that a shared resource, for example, a printer, can only be accessed by one process at a time.

Critical Section Problem:

- We know that Critical Section is a code segment that accesses shared variables and has to be executed as an atomic operation or atomic action.
- It means that in a group of cooperating processes, at a given point of time, only one process must be executing its critical section. If any other process also wants to execute its critical section, it must wait until the first one finishes. The entry to the critical section is mainly handled by wait() function while the exit from the critical section is controlled by the signal() function.



Entry Section:

In this section mainly the process requests for its entry in the critical section. It is controlled by wait() function.

Exit Section:

This section is followed by the critical section that represents exit from critical section. It is handled by signal() function.

Conditions must satisfy for the solution of the Critical Section Problem:

A solution to the critical section problem must satisfy the following three conditions:

1. Mutual Exclusion:

Out of a group of cooperating processes, only one process can be in its critical section at a given point of time.

2. Progress:

If no process is in its critical section, and if one or more threads want to execute their critical section then any one of these threads must be allowed to get into its critical section.

3. Bounded Waiting:

After a process makes a request for getting into its critical section, there is a limit for how many other processes can get into their critical section, before this process's request is granted. So after the limit is reached, the system must grant the process permission to get into its critical section.

Solutions for the Critical Section Problem:

The critical section plays an important role in Process Synchronization so that the problem must be solved. Some widely used methods to solve the critical section problem are as follows:

- | | |
|------------------------|------------------------------|
| 1) Peterson's Solution | 2) Synchronization Hardware. |
| 3) Mutex Locks | 4) Semaphore |

Let's see these methods in details-

1) Peterson's Solution:

- This is widely used and software-based solution to critical section problems. Peterson's solution was developed by a computer scientist Gary L. Peterson in 1981 that's why it is named so.
- With the help of this solution whenever a process is executing in any critical state, then the other process only executes the rest of the code.
- This method also helps to make sure that only a single process can run in the critical section at a specific time.
- Peterson's algorithm (or Peterson's solution) is a concurrent programming algorithm for mutual exclusion that allows two or more processes to share a single-use resource without conflict, using only shared memory for communication.
- While Peterson's original solution/algorithm/formulation worked with only two processes, the algorithm can be generalized for more than two processes but here we consider for only two processes.
- **The algorithm-**
 - This algorithm uses two variables, flag and turn related with each process
 - A flag[n] value of 'true' indicates that the process n wants to enter the critical section.
 - turn specifies that whose turn is it to enter into critical section.
 - Entrance to the critical section is granted for process P0 If-
 - ❖ P1 does not want to enter its critical section or
 - ❖ P1 has given chance to P0 to go into critical section by setting turn to 0.

Structure of Peterson algorithm-

```
bool    flag[2] = {false, false};
int      turn;
```

P0: flag[0] = true; turn = 1; while (flag[1] == true && turn == 1) { // busy wait }  flag[0] = false;	P1: flag[1] = true; turn = 0; while (flag[0] == true && turn == 0) { // busy wait }  flag[1] = false;
--	---

- This solution preserves all following three conditions:
 - Mutual Exclusion is supported (comforted) as at any time only one process can access the critical section. i.e. turn cannot be 0 and 1 at a same time.
 - Progress is also comforted, as a process that is outside the critical section is unable to block other processes from entering into the critical section.
 - Bounded Waiting is also assured as every process gets a fair chance to enter the Critical section.

2) Synchronization Hardware:

- Many systems provide hardware support for critical section code. The critical section problem could be solved easily in a single-processor environment if we disallow interrupts to occur while a shared variable or resource is being modified.
- In this manner, we could be sure that the current sequence of instructions would be allowed to execute in order without pre-emption. Unfortunately, this solution is not feasible in a multiprocessor environment.
- Disabling interrupt on a multiprocessor environment can be time-consuming as the message is passed to all the processors.
- This message transmission lag delays the entry of threads into the critical section, and the system efficiency decreases. Thus, the synchronization hardware solution is not easy to implement for everyone.

3) Mutex Locks:

- As the synchronization hardware solution is not easy to implement for everyone, a strict software approach called 'Mutex Locks' was introduced.
- In this approach, in the entry section of code, a LOCK is acquired over the critical section resources and used inside the critical section, and in the exit section that LOCK is released.
- As the resource is locked while a process executes its critical section hence no other process can access it.

4) Semaphore:

- Semaphore is also software-based solution over the critical section problem which was proposed by Dijkstra in 1965 which is a very significant technique to manage concurrent processes by using a simple integer variable, which is known as a 'semaphore'.
- Semaphore is simply an integer variable that is non-negative and shared between threads. This variable is used to solve the critical section problem and to achieve process synchronization in the multiprocessing environment.
- In short, semaphore is protected integer variable that can help and restrict access to shared resources in multiprocessing environment.
- In Semaphores, the critical section problem is solved or process synchronization is achieved by using two atomic operations, wait() and signal() that are used for process synchronization.

The definitions of wait and signal are as follows –

1. Wait() - [P() or down() or sleep()]

- Wait() is called when a process want access to resources (when semaphore variable is negative, the process wait is blocked). It is used in entry section of process code.
- The wait operation decrements the value of its argument S, if it is positive. If S is negative or zero, then no operation is performed i.e. process/thread goes in waiting state or blocked.

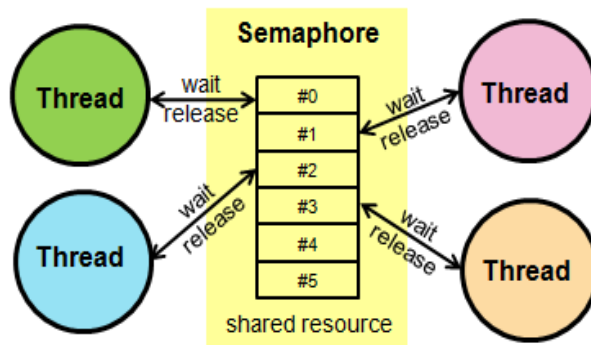
```
wait(S)
{
    while (S<=0)
    {
        // wait or suspend
    }
    S--;
}
```

2. Signal() - [V() or up() or wakeup()]

- Signal() is called after a process executes its critical section.
- It is used in exit section of process code.
- The signal operation increments the value of its argument S.

```
signal(S)
{
    S++;
}
```

Following diagram shows semaphore:



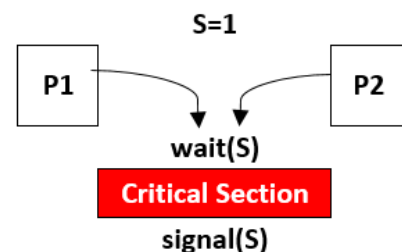
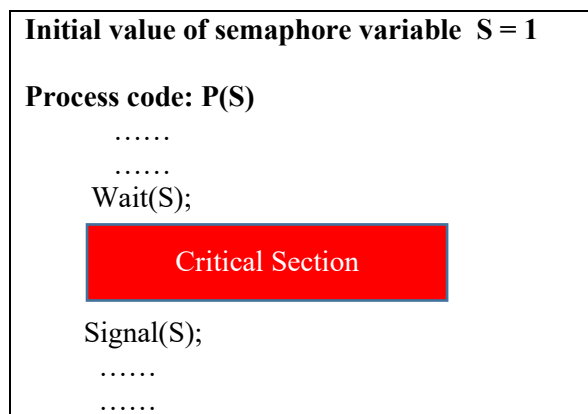
Types of Semaphores:

Semaphores are of two types namely- 1) Binary Semaphores 2) Counting Semaphores

Let's see these in details-

1) Binary Semaphores:

- The binary semaphore is integer variable that holds only two values either 0 or 1.
- The wait operation only works when the semaphore is 1 and the signal operation succeeds when semaphore is 0.
- It is sometimes easier to implement binary semaphores than counting semaphores.
- Let's consider following process code example,
- If there are two process P1 and P2 are trying to enter in critical section as shown in following figure:

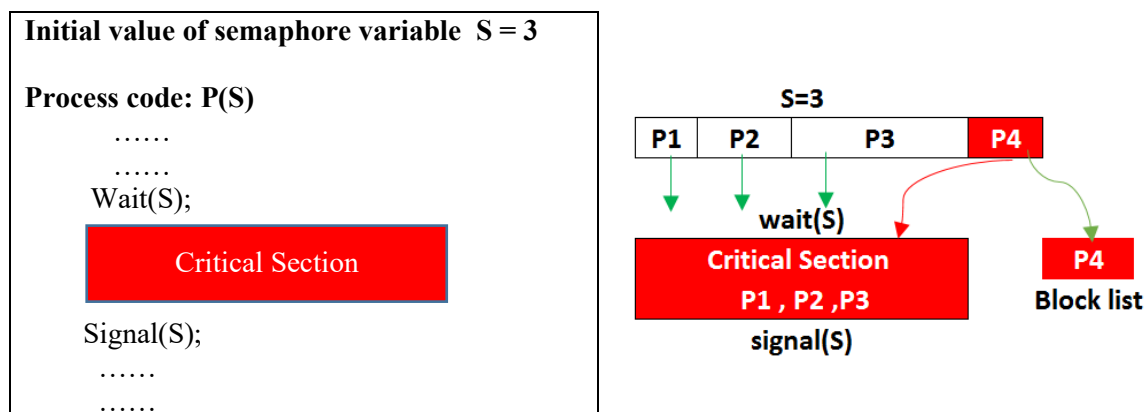


- Before entering into critical section, each process has to execute process code sequentially as shown above.
- By executing the code by the processes it is clear that no two process can enter simultaneously into critical section, here mutual exclusion is satisfied.

2) Counting Semaphores:

- Counting semaphores are integer variable semaphores and have an unrestricted value domain i.e it would hold more than two values from $-\infty$ to ∞ that depends on total available instances or resources.
- Counting semaphores are used to coordinate the resource access, where the semaphore count is the number of available resources.
- If the resources are added, semaphore count automatically incremented and if the resources are removed, the count is decremented.
- Let's again consider following process code,
- Here, initially we assign value for $S=3$ it means a system has 3 units or instances of resources.
- One by one maximum three processes P1, P2, P3 allows to enter in critical section (Since, value of semaphore variable is 3)

- If extra other processes are trying to enter in critical section then they are blocked or suspended till any other previous process come out from critical section and execute signal() shown in following figure:



- The negative value of semaphore indicate that how many processes are in blocked or suspended state.

Advantages of Semaphores:

Some of the advantages of semaphores are as follows –

- 1) Semaphores (In case of binary semaphore) allow only one process into the critical section. They follow the mutual exclusion principle strictly and are much more efficient than some other methods of synchronization.
- 2) Semaphores are implemented in the machine independent code of the microkernel. So they are machine independent.
- 3) The resource used in critical section always remains busy because of busy waiting schedule used by semaphores.

Disadvantages of Semaphores:

Some of the disadvantages of semaphores are as follows –

- 1) The wait and signal operations must be implemented in the correct order in semaphore to prevent deadlocks.
- 2) Semaphores may lead to a priority inversion where low priority processes may access the critical section first and high priority processes later.
- 3) We know that semaphores uses wait() and signal() operations and these operation prevents the creation of a structured layout for the system therefore semaphores leads to loss of modularity.

Classical Problems of Synchronization:

Below are some of the classical problems showing the process synchronization in systems where cooperating processes are present.

- 1) Bounded Buffer (Producer-Consumer) Problem
- 2) Dining Philosophers Problem
- 3) The Readers Writers Problem

We will discuss the above three problems as follow-

1) Producer-consumer Problem (Bounded Buffer Problem):

- The producer consumer problem is a synchronization problem. There is a fixed size buffer and the producer produces items and enters them into the buffer. The consumer removes the items from the buffer and consumes them.
- A producer should not produce items into the buffer when the consumer is consuming an item from the buffer and vice versa. So the buffer should only be accessed by the producer or consumer at a time.
- The producer consumer problem can be resolved using semaphores. The codes for the producer and consumer process are given as follows –

Producer Process:

The code that defines the producer process is given below –

```
do {  
    .  
    . PRODUCE ITEM  
    .  
    wait(empty);  
    wait(mutex);  
    PUT ITEM IN BUFFER  
    signal(mutex);  
    signal(full);  
} while(1);
```

- In the above code, mutex, empty and full are semaphores. Here mutex is initialized to 1, empty is initialized to n (maximum size of the buffer) and full is initialized to 0.
- The mutex semaphore ensures mutual exclusion. The empty and full semaphores count the number of empty and full spaces in the buffer.
- After the item is produced, wait operation is carried out on empty. This indicates that the empty space in the buffer has decreased by 1. Then wait operation is carried out on mutex so that consumer process cannot interfere.
- After the item is put in the buffer, signal operation is carried out on mutex and full. This indicates that consumer process can now act and also shows that the buffer is full by 1.

Consumer Process:

The code that defines the consumer process is given below:

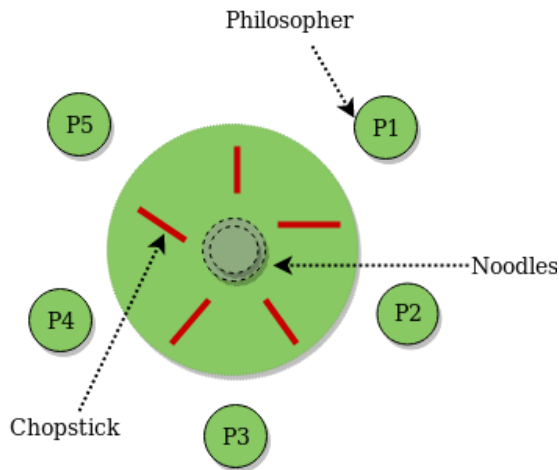
```
do {  
    wait(full);  
    wait(mutex);  
    REMOVE ITEM FROM BUFFER  
    signal(mutex);  
    signal(empty);  
    .  
    . CONSUME ITEM  
    .  
} while(1);
```

- The wait operation is carried out on full. This indicates that items in the buffer have decreased by 1. Then wait operation is carried out on mutex so that producer process cannot interfere.
- Then the item is removed from buffer. After that, signal operation is carried out on mutex and empty. This indicates that producer process can now act and also shows that empty space in the buffer has increased by 1.

2) Dining Philosophers Problem:

- **Problem-** The dining philosopher's problem states that there are 5 philosophers sharing a circular dining table and they eat noodles and think alternatively. There is a bowl of Noodles for the philosophers and 5 chopsticks. A philosopher needs both their right and left chopstick to eat. A philosopher may only eat if there are both chopsticks available. Otherwise a philosopher puts down their chopstick and begin thinking again.
- The dining philosopher is a classic synchronization problem as it demonstrates a large class of concurrency control problems.

Fig-



Solution of Dining Philosophers Problem:

- A solution of the Dining Philosophers Problem is to use a semaphore to represent a chopstick. A chopstick can be picked up by executing a wait operation on the semaphore and released by executing a signal operation of semaphore.
- The structure of the chopstick is shown below –
Semaphore chopstick [5];
- Initially the elements of the chopstick are initialized to 1 it indicates that chopsticks are on the table and not picked up by a philosopher.
- The structure of a random philosopher 'i' is given as follows –

```
do {
    wait( chopstick[i] );
    wait( chopstick[ (i+1) % 5] );
    EATING THE Noodles operation
    signal( chopstick[i] );
    signal( chopstick[ (i+1) % 5] );
    .
    . THINKING
    .
} while(1);
```

- In the above structure, first wait operation is performed on chopstick[i] and chopstick[(i+1) % 5]. This means that the philosopher 'i' has picked up the chopsticks on his sides. Then the eating function is performed.
- After that, signal operation is performed on chopstick[i] and chopstick[(i+1) % 5]. This means that the philosopher 'i' has eaten and put down the chopsticks on his sides. Then the philosopher goes back to thinking.

Difficulty with the solution:

The above solution makes sure that no two neighboring philosophers can eat at the same time. But this solution can lead to a deadlock. This may happen if all the philosophers pick their left chopstick simultaneously. Then none of them can eat and deadlock occurs.

Some of the ways to avoid deadlock are as follows –

- There should be at most four philosophers on the table.
- A philosopher should only be allowed to pick their chopstick if both are available at the same time.
- An even philosopher should pick the right chopstick and then the left chopstick while an odd philosopher should pick the left chopstick and then the right chopstick.

3) The Readers Writers Problem:

- In this problem there are some processes (called readers) that only read the shared data, and never change it, and there are other processes (called writers) who may change the shared data in addition to reading.
- There are various type of readers-writers problem, most can run on relative priorities of readers and writers.
- The main complexity with this problem occurs for allowing more than one reader/writer process to access the shared data at the same time.

The Problem Statement:

- There are two types of processes in this context. One is reader process and another is writer process.
- There is a shared resource which should be accessed by multiple processes.
- Any number of readers can read from the shared resource simultaneously, but only one writer can write to the shared resource.
- When a writer process is writing data to the resource, no other process can access the resource. A writer process cannot write to the resource if any reader process accessing the resource at that time.

The Solution:

- From the above problem statement, it shows that readers process have higher priority than writer. If a writer wants to write to the resource, it must wait until there are no readers currently accessing that resource.
- Here, we use one mutex lock 'm' and one semaphore 'w'. Initially value 1 is assigned to 'm' and 'w'.
- We use one integer variable *read_count* which is used to count the number of readers currently accessing the resource. The variable *read_count* is initialized to 0.
- We use the mutex lock 'm' to make the process to acquire and release lock on shared resources whenever it is updating the *read_count* variable.

Code for the writer process:

```
while(TRUE)
{
    wait(w);

    Perform write operation

    signal(w);
}
```

Code for the reader process:

```
while(TRUE)
{
    wait(m); //acquire lock
    read_count++;
    if(read_count == 1)
    {
        wait(w);
    }
    signal(m); //release lock

    Perform Read operation

    wait(m); // acquire lock
    read_count--;
    if(read_count == 0)
    {
        signal(w);
    }
    signal(m); // release lock
}
```

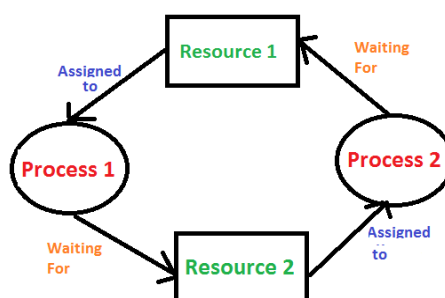
Explanation of above Code:

- As seen above in the code for the writer, the writer just waits on the 'w' semaphore until it gets a chance to write to the resource.
- After performing the write operation, it increments 'w' so that the next writer can access the resource.
- On the other hand, in the code for the reader, the lock is acquired whenever the *read_count* is updated by a process.
- When a reader wants to access the resource, first it increments the *read_count* value, then accesses the resource and then decrements the *read_count* value.

- The semaphore 'w' is used by the first reader which enters the critical section and the last reader which exits the critical section.
- The reason for this is, when the first readers enters the critical section, the writer is blocked from the resource. Only new readers can access the resource now.
- Similarly, when the last reader exits the critical section, it signals the writer using the 'w' semaphore because there are zero readers now and a writer can have the chance to access the resource.

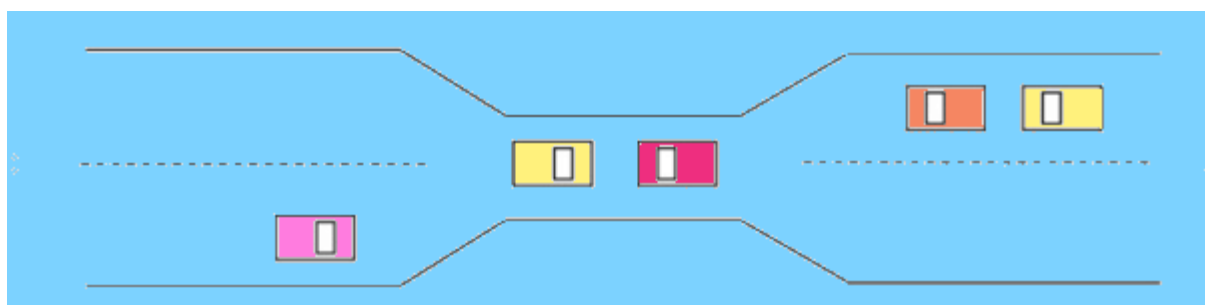
Deadlocks:

- We know that, program in execution is called 'Process' and a process in operating systems uses different resources in following way:
 - 1) Requests a resource.
 - 2) Use the resource.
 - 3) Releases the resource.
- A deadlock happens in operating system when two or more processes needs some resource to complete their execution but that resource is already held by the other process.
- Deadlock is a situation that occurs in OS when any process enters a waiting state because another process is holding the demanded resource. Deadlock is a common problem in multi-processing where several processes share a specific type of mutually exclusive resource known as a 'soft lock' or 'software.'
- Or we can say that Deadlock is a situation where a set of processes are blocked (or in waiting state) because each process is wants to access a resource which is already acquired by some other process.
- Example- When two trains are coming toward each other on the same track and there is only one track, none of the trains can move back once they are in front of each other. A similar situation occurs in operating systems when there are two or more processes that hold some resources and wait for resources held by other(s). For example, in the below diagram, Process 1 is holding Resource 1 and waiting for resource 2 which is acquired by process 2, and process 2 is waiting for resource 1.



A real-world example of deadlock would be traffic found on a narrow bridge is available.

- Here, a bridge is considered a resource.
- So, when Deadlock happens, it can be easily resolved if one car backs up (Preempt resources and rollback).
- Several cars may have to be backed up if a deadlock situation occurs.
- So starvation is possible.



Deadlock Characterization: (Necessary Conditions for deadlock)

- We know that a deadlock happens in operating system when two or more processes need some resource to complete their execution that is held by the other process.
- A deadlock occurs if the four conditions (Mutual exclusion, hold and wait, no preemption, circular wait) are true. They are given as follows –

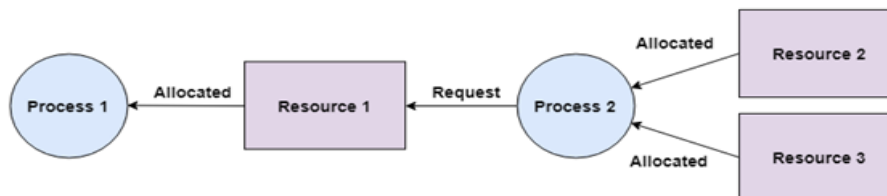
1) Mutual Exclusion:

- There should be a resource that can only be held by one process at a time. In the diagram below, there is a single instance of Resource 1 and it is held by Process 1 only.



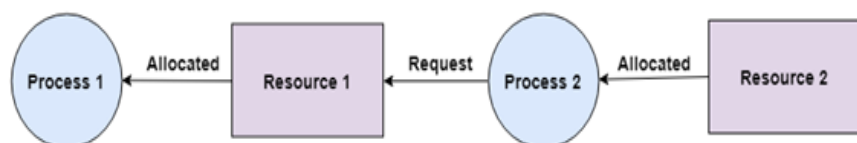
2) Hold and Wait:

- A process can hold multiple resources and still request more resources from other processes which are holding them to complete its execution. In the diagram given below, Process 2 holds Resource 2 and Resource 3 and is requesting the Resource 1 which is held by Process 1.



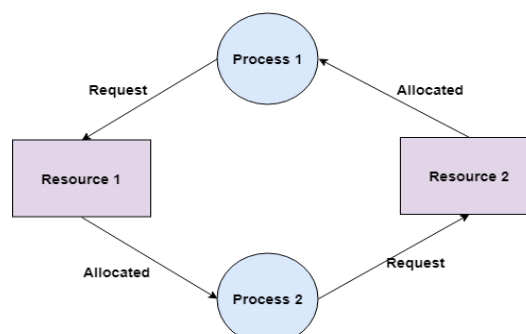
3) No Preemption:

- A resource cannot be preempted from a process by force. A process can only release a resource voluntarily (own free). In the diagram below, Process 2 cannot preempt Resource 1 from Process 1. It will only be released when Process 1 relinquishes it voluntarily after its execution is complete.



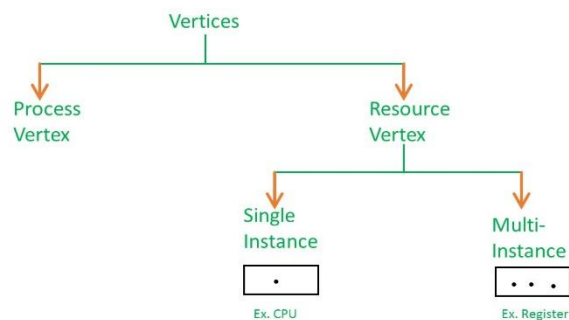
4) Circular Wait:

- A process is waiting for the resource held by the second process, which is waiting for the resource held by the third process and so on, till the last process is waiting for a resource held by the first process. This forms a circular wait. For example: Process 1 is allocated Resource2 and it is requesting Resource 1. Similarly, Process 2 is allocated Resource 1 and it is requesting Resource 2. This forms a circular wait loop.



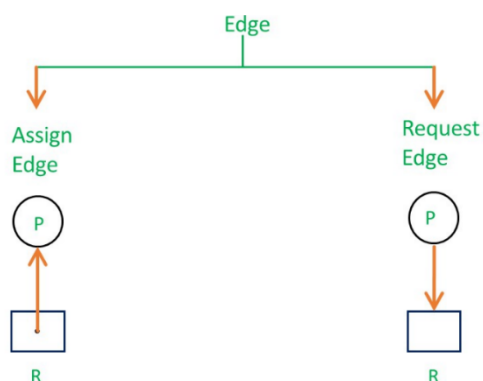
Resource allocation graph: (RAG)

- The resource allocation graph is the pictorial representation of the state of a system. As its name suggests, the resource allocation graph is the complete information about all the processes which are holding some resources or waiting for some resources.
- It also contains the information about all the instances of all the resources whether they are available or being used by the processes.
- Resource allocation graph is explained to us- what is the state of the system in terms of processes and resources. Like how many resources are available, how many are allocated and what is the request of each process. Everything can be represented in terms of the diagram and that diagram is nothing but RAG.
- One of the advantages of RAG, it is possible to see a deadlock directly.
- We know that any graph contains vertices and edges. So RAG also contains vertices and edges. In RAG vertices are two type –
 1. **Process vertex** – Every process in a system is represented by a process vertex. Generally, the process will be represented with a circle symbol.
 2. **Resource vertex** – Every resource in a system is represented by resource vertex. Generally, the resource will be represented with a rectangle or box symbol.It has two types –
 - Single instance resource vertex– It represents as a box or rectangle. Inside the box, there will be one dot.
 - Multi-instance resource vertex– It also represents as a box or rectangle. Inside the box, there will be many dots present. So the number of dots indicates how many instances are present of each resource type.
- Following diagram shows types of vertices in RAG.



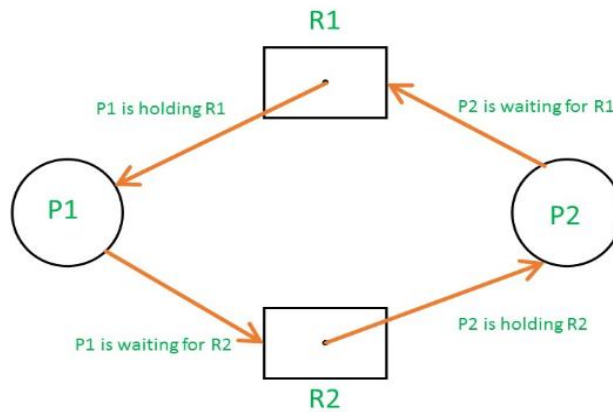
Now coming to the edges of RAG. There are two types of edges in RAG –

1. **Assign Edge** – If you already assign a resource to a process then it is called “Assign edge”.
2. **Request Edge** – It means in future the process might want some resource to complete the execution that is called “request edge”.



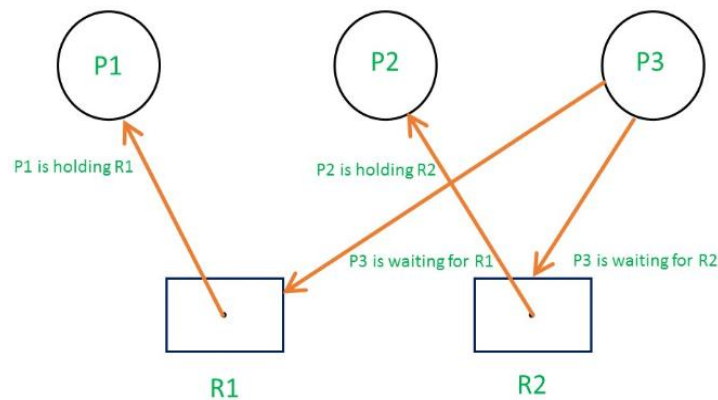
So, If a process is using a resource, an arrow is drawn from the resource vertex to the process vertex.
 If a process is requesting a resource, an arrow is drawn from the process node to the resource node.

Example 1: (Single instances RAG) –



SINGLE INSTANCE RESOURCE TYPE WITH DEADLOCK

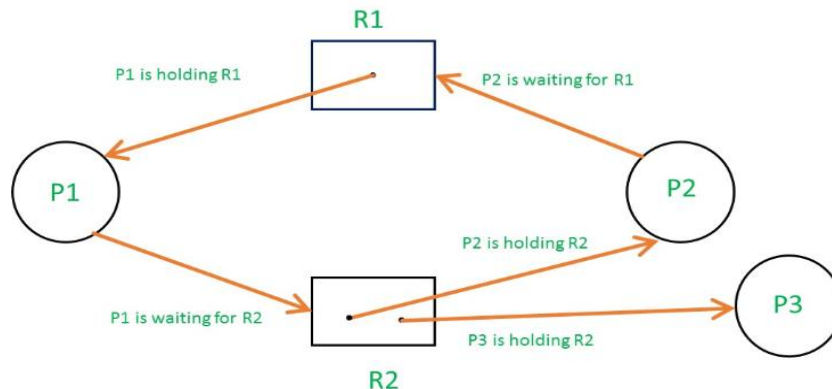
If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will be in deadlock. For example, if process P1 holds resource R1, process P2 holds resource R2 and process P1 is waiting for R2 and process P2 is waiting for R1, then process P1 and process P2 will be in deadlock.



SINGLE INSTANCE RESOURCE TYPE WITHOUT DEADLOCK

Here's another example, which shows Processes P1 and P2 acquiring resources R1 and R2 while process P3 is waiting to acquire both resources. In this example, there is no deadlock because there is no circular dependency. So cycle in single-instance resource type is the sufficient condition for deadlock.

Example 2 (Multi-instances RAG) –



MULTI INSTANCES WITHOUT DEADLOCK

Note: In above diagram, there is no circular wait between process P1, P2 and P3 therefore deadlock not happen.

Methods of Handling Deadlocks:

- We know that, Deadlock is a situation where a process or a set of processes is blocked or waiting for some other resource that is held by some other waiting process. It is an undesirable (unwanted) state of the system.
- There are four conditions mutual exclusion, hold and wait, no preemption and circular wait that must hold simultaneously for a deadlock to occur.

There are four strategies or methods to handle the deadlocks:

- 1) Deadlock Ignorance.
- 2) Deadlock Prevention.
- 3) Deadlock Avoidance.
- 4) Deadlock detection & recovery.

Let's see these methods in details-

1) Deadlock Ignorance:

- Deadlock Ignorance is the most widely used approach among all the mechanism. This is being used by many operating systems mainly for end user uses.
- In this approach, the Operating system assumes that deadlock never occurs. It simply ignores deadlock. This approach is best suitable for a single end user system where User uses the system only for browsing and all other normal stuff.
- The operating systems like Windows and Linux mainly focus upon performance. However, the performance of the system decreases if it uses deadlock handling mechanism all the time if deadlock happens 1 out of 100 times then it is completely unnecessary to use the deadlock handling mechanism all the time.
- In these types of systems, the user has to simply restart the computer in the case of deadlock. Windows and Linux are mainly using this approach.

2) Deadlock Prevention:

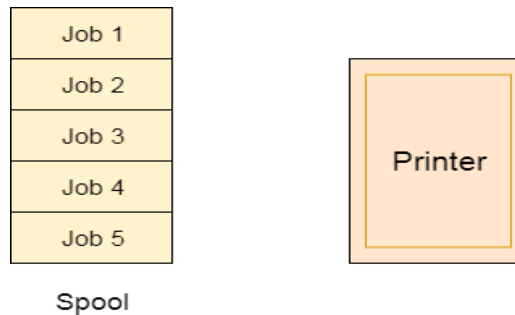
- We know that, deadlock happens only when Mutual Exclusion, hold and wait, No preemption and circular wait holds simultaneously.
- If it is possible to violate or prevent one of the four conditions at any time and don't let them occur together then we can prevent the deadlock.
- The idea behind the approach is very simple that we have to fail one of the four conditions but there can be a big challenge on its physical implementation in the system.
- Let's see how we can prevent each of the conditions.

1. Mutual Exclusion:

- We know that in mutual exclusion, only one resource is allocated to only one process at a time and this is the main reason behind the deadlock. If a resource could have been used by more than one process at the same time then the process would have never been waiting for any resource.
- However, if we can be able to violate resources behaving in the mutually exclusive manner then the deadlock can be prevented.

Spooling:

- For a device like printer, spooling can work. There is a memory associated with the printer which stores jobs from each of the process into it. Later, Printer collects all the jobs and print each one of them according to FCFS. By using this mechanism, the process doesn't have to wait for the printer and it can continue whatever it was doing. Later, it collects the output when it is produced.



Although, Spooling can be an effective approach to violate mutual exclusion but it suffers from two kinds of problems.

1. This cannot be applied to every resource.
2. After some point of time, there may arise a race condition between the processes to get space in that spool.
- We cannot force a resource to be used by more than one process at the same time since it will not be fair enough and some serious problems may arise in the performance. Therefore, we cannot violate mutual exclusion for a process practically.

2. Hold and Wait:

- We know that, Hold and wait condition occurs when a process holds a resource and waiting for some other resource to complete its task. Deadlock occurs because there can be more than one process which are holding one resource and waiting for other in the cyclic order.
- However, we have to find out some mechanism by which a process either doesn't hold any resource or doesn't wait. That means, a process must be assigned all the necessary resources before the execution starts. A process must not wait for any resource once the execution has been started.

!(Hold and wait) = !hold or !wait (negation of hold and wait is, either you don't hold or you don't wait)

- This can be implemented practically if a process declares all the resources initially. However, this sounds very practical but can't be done in the computer system because a process can't determine necessary resources initially.
- Process is the set of instructions which are executed by the CPU. Each of the instruction may demand multiple resources at the multiple times. The need cannot be fixed by the OS.

The problem with the approach is:

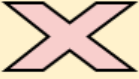
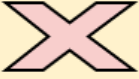
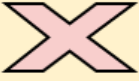
1. Practically not possible.
2. Possibility of getting starvation due to the fact that some process may hold a resource for a very long time.

3) No preemption:

- We know that, once a resource is allocated to a process then we cannot force fully preempts that resource from a process & that causes deadlock. However, if we take the resource away (preempt) from the process then we can prevent deadlock.
- This is not a good approach at all since if we take a resource away which is being used by the process then all the work which it has done till now can become inconsistent.
- Consider a printer is being used by any process. If we take the printer away from that process and assign it to some other process then all the data which has been printed can become inconsistent and effective less and also the fact that the process can't start printing again from where it has left last time.

4) Circular Wait:

- To violate circular wait, we have to break the circular dependency of resources used by processes for that we can assign a priority number to each of the resource.
- Here, a process requests for a resource by incremental order of priorities of resources that causes circular wait doesn't happen. Note that, a process can't request for a lesser priority resource. This ensures that a process cannot request a resource which is being utilized by some other process and no cycle will be formed.

Condition	Approach	Is Practically Possible?
Mutual Exclusion	Spooling	
Hold and Wait	Request for all the resources initially	
No Preemption	Snatch all the resources	
Circular Wait	Assign priority to each resources and order resources numerically	

Among all the methods, violating Circular wait is the only approach that can be implemented practically. Thus, we can prevent the deadlock.

3) Deadlock Avoidance:

- In deadlock avoidance, at every step the operating system checks whether the system is in safe state or in unsafe state. The process continues until the system is in safe state. Once the system moves to unsafe state, the OS has to backtrack one step.
- In simple words, The OS continue reviews for each resource allocation so that the allocation doesn't cause the deadlock in the system.
- In deadlock avoidance, the request for any resource will be granted if the resulting state of the system doesn't cause deadlock in the system. The state of the system will continuously be checked for safe and unsafe states.
- In order to avoid deadlocks, the process must tell OS, the maximum number of resources a process can request to complete its execution.
- The simplest and most useful approach states that the process should declare the maximum number of resources of each type it may ever need.
- The Deadlock avoidance algorithm examines the resource allocations for the process so that there can never be a circular wait condition.
- **Resource allocation state (Safe and Unsafe States):**
- The resource allocation state of a system can be defined by available and allocated instances of resources, and the maximum instance of the resources demanded by the processes.
- A state of a system recorded at some random time is shown below.

Resources Assigned:

Process	Type 1	Type 2	Type 3	Type 4
A	3	0	2	2
B	0	0	1	1
C	1	1	1	0
D	2	1	4	0

Resources still needed to complete execution:

Process	Type 1	Type 2	Type 3	Type 4
A	1	1	0	0
B	0	1	1	2
C	1	2	1	0
D	2	1	1	2

E = (7 6 8 4) (Let, It is initially considered in system)

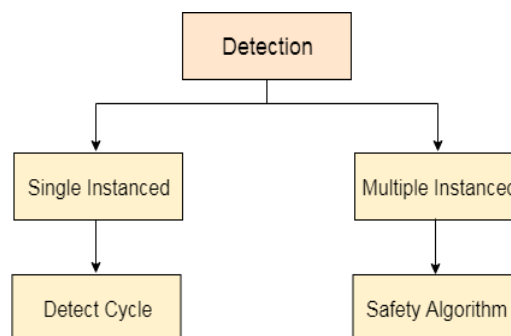
$$P = (6 \ 2 \ 8 \ 3)$$

$$A = (1 \ 4 \ 0 \ 1)$$

- Above tables and vector E, P and A describes the resource allocation state of a system. There are 4 processes and 4 types of the resources in a system.
- Table 1 shows the instances of each resource assigned to each process.
- Table 2 shows the instances of the resources that each process still needs.
- Vector E represents total number of instances of each resource in the system.
- Vector P represents the instances of resources that have been assigned to processes.
- Vector A represents the number of resources that are not in use.
- A state of the system is called 'safe' if the system can allocate all the resources requested by all the processes without entering into deadlock.
- If the system cannot fulfill the request of all processes then the state of the system is called 'unsafe'.
- The key of Deadlock avoidance approach is when the request is made for resources then the request must only be approved in the case if the resulting state is also a safe state.
- If system is in unsafe state & resource allocation request is granted then it again leads to 'Deadlock' therefore in deadlock avoidance, resource allocation request is granted only if system is in 'Safe' state such that deadlock never occurs.

4) Deadlock Detection and Recovery:

- This approach let the processes fall in deadlock and then periodically check whether deadlock occur in the system or not. If it occurs then it applies some of the recovery methods on the system to resolve deadlock.
- In this approach, The OS doesn't apply any mechanism to avoid or prevent the deadlocks. Therefore the system considers that the deadlock will definitely occur. In order to get rid of deadlocks, The OS periodically checks the system for any deadlock. In case, it finds any of the deadlock then the OS will recover the system using some recovery techniques.
- The main task of the OS is detecting the deadlocks. And we know that, OS can detect the deadlocks with the help of Resource allocation graph.



- In single instance resource types, if a cycle is being formed in the system then there will definitely be a deadlock.
- On the other hand, in multiple instanced resource type graph, detecting a cycle is not just enough. We have to apply the safety algorithm on the system by converting the resource allocation graph into the allocation matrix and request matrix.
- In order to **recover** the system from **deadlocks**, either OS considers resources or processes (Not both considered at a time)

Deadlock recovery by considering Resource-

➤ Preempt the resource:

We can snatch or take one of the resources from the owner of the resource (process) and give it to the other process with the expectation that it will complete the execution and will release this resource sooner. Well, choosing a resource which will be snatched is going to be a bit difficult.

➤ **Rollback to a safe state:**

- We know that, a system passes through various states to get into the deadlock state. The operating system can roll back the system to the previous safe state. For this purpose, OS needs to implement check pointing at every state.
- The moment, we get into deadlock, we will rollback all resource allocations to get into the previous safe state.

Deadlock recovery by considering Process-

➤ **Kill a process:**

Killing a process can solve our deadlock problem but the bigger concern is to decide which process to kill. Generally, Operating system kills a process which has done least amount of work until now.

➤ **Kill all process:**

This is not a suggestible approach but can be implemented if the problem becomes very serious. Killing all process will lead to inefficiency in the system because all the processes will execute again from starting.

